

```
In [92]: import numpy as np
import pandas as pd

import scipy.stats as stats
from scipy.stats import kstest, norm, shapiro, anderson, jarque_bera, probpl

from statsmodels.stats.proportion import proportion_confint
from scipy.special import beta, gamma, gammaln, betaln

from scipy.optimize import minimize

import matplotlib.pyplot as plt
import seaborn as sns
```

Task 1

We start with the verification of the law of large numbers and the central limit theorem. Thus, we check if the mean $\bar{X}$ converges (in probability) to the expectation μ as the sample size increases, and if its distribution converges to the normal distribution.

Part A

- Simulate samples of size $n = 5, 5^2, 5^3, \dots, 5^8$ from a normal distribution with mean 1 and variance 1, i.e., $N(1, 1)$.
- For each sample, compute the sample mean and the sample variance.
- Plot the path of the sample means as a function of n (use a log scale on the x-axis).
- **Question:** What conclusion can we draw from the figure if we keep in mind the law of large numbers?

```
In [ ]: # Set seed for reproducibility
np.random.seed(0)

# Define sample sizes as powers of 5
sample_sizes = [5**i for i in range(1, 9)]

# Store means and variances
sample_means = []
sample_variances = []

# Loop over sample sizes
for n in sample_sizes:
    # Generate sample of size n from N(1, 1)
    sample = np.random.normal(1, 1, n)

    # Compute sample mean and variance
```

```

mean = np.mean(sample)
variance = np.var(sample, ddof=1)

# Store results
sample_means.append(mean)
sample_variances.append(variance)

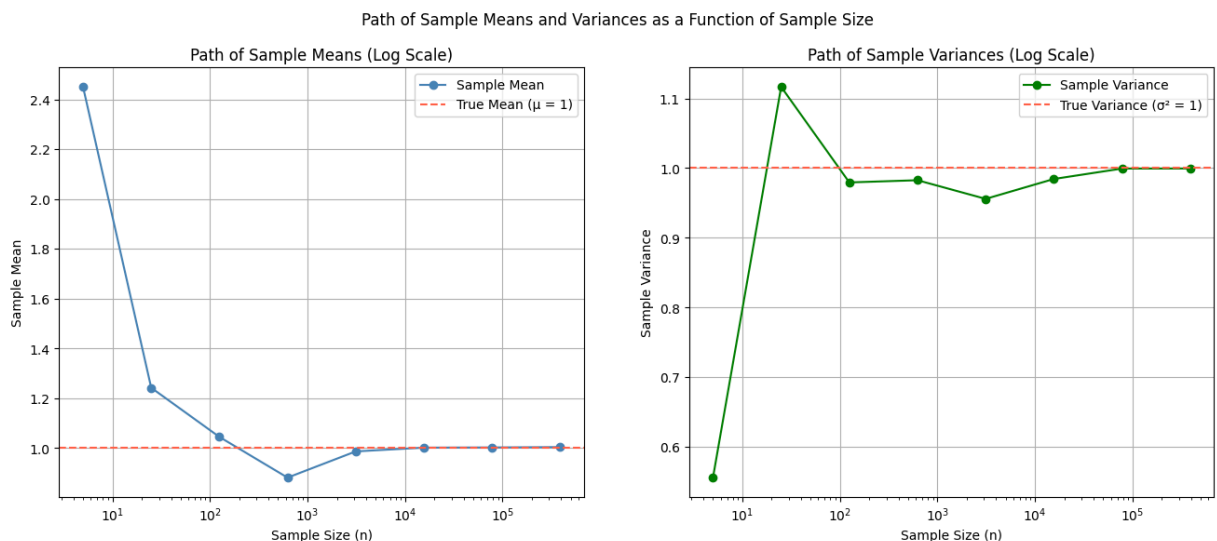
# Plotting the path of sample means and variances as a function of sample size
fig, axs = plt.subplots(1, 2, figsize=(16, 6))

# Plot for sample means
axs[0].plot(sample_sizes, sample_means, marker='o', linestyle='--', color='steelblue')
axs[0].set_xscale('log')
axs[0].set_xlabel('Sample Size (n)')
axs[0].set_ylabel('Sample Mean')
axs[0].set_title('Path of Sample Means (Log Scale)')
axs[0].axhline(y=1, color='tomato', linestyle='--', label='True Mean ( $\mu = 1$ )')
axs[0].legend()
axs[0].grid(True)

# Plot for sample variances
axs[1].plot(sample_sizes, sample_variances, marker='o', linestyle='--', color='green')
axs[1].set_xscale('log')
axs[1].set_xlabel('Sample Size (n)')
axs[1].set_ylabel('Sample Variance')
axs[1].set_title('Path of Sample Variances (Log Scale)')
axs[1].axhline(y=1, color='tomato', linestyle='--', label='True Variance ( $\sigma^2 = 1$ )')
axs[1].legend()
axs[1].grid(True)

plt.suptitle('Path of Sample Means and Variances as a Function of Sample Size')
plt.show()

```



Law of Large Numbers (LLN)

The Law of Large Numbers (LLN) states that as the sample size n increases, the sample mean $\bar{X}$ converges in probability to the expected value μ . This can be expressed as:

$$\lim_{n \rightarrow \infty} P\left(\left|\bar{X} - \mu\right| < \epsilon\right) = 1$$

for any $\epsilon > 0$.

Analysis of Sample Means and Variances

- **Sample Mean:** In the left plot, the sample mean $\bar{X}$ fluctuates for small n but stabilizes near the true mean $\mu = 1$ as n grows, demonstrating the LLN.
- **Sample Variance:** The right plot shows that the sample variance also converges to the true variance $\sigma^2 = 1$ with increasing n , supporting the consistency of sample variance.

Central Limit Theorem (CLT) Justification

By the Central Limit Theorem, for large n , the sample mean $\bar{X}$ approaches a normal distribution:

$$\bar{X} \sim N\left(\mu, \frac{\sigma^2}{n}\right)$$

As $n \rightarrow \infty$, the variance $\frac{\sigma^2}{n}$ approaches zero, meaning $\bar{X}$ concentrates around μ .

Conclusion

These results confirm the Law of Large Numbers:

1. The sample mean $\bar{X}$ converges to the true mean $\mu = 1$.
2. The sample variance converges to $\sigma^2 = 1$.

Thus, as n increases, both the sample mean and variance become accurate estimates of the population parameters, validating the LLN.

Part B

- For each sample, compute manually the 95% confidence interval for the mean and add it to the plot.
- Perform this with both known σ and estimated σ .
- **Question:** Discuss what you observe and provide an explanation for your findings.

```
In [ ]: # Known population standard deviation (sigma) and confidence level
sigma = 1
confidence_level = 0.95
```

```

# Calculate the critical z-value for 95% confidence
z_critical = stats.norm.ppf((1 + confidence_level) / 2)

# Lists to store confidence intervals
ci_known_sigma = []
ci_estimated_sigma = []

# Loop over each sample size to calculate confidence intervals
for i, n in enumerate(sample_sizes):
    # Sample mean for this sample size
    mean = sample_means[i]

    # Known sigma confidence interval
    margin_of_error_known = z_critical * (sigma / np.sqrt(n))
    ci_known_sigma.append((mean - margin_of_error_known, mean + margin_of_er

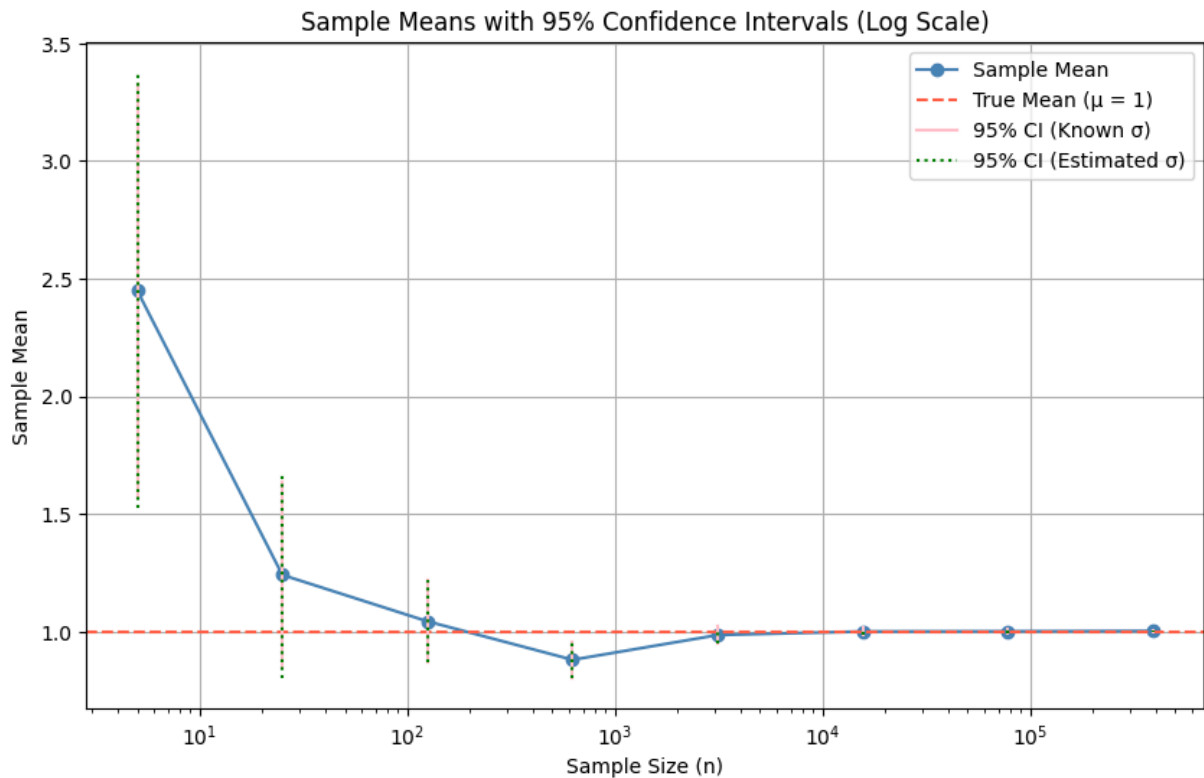
    # Estimated sigma (sample standard deviation) confidence interval
    estimated_sigma = np.sqrt(sample_variances[i])
    t_critical = stats.t.ppf((1 + confidence_level) / 2, df=n-1)
    margin_of_error_estimated = t_critical * (estimated_sigma / np.sqrt(n))
    ci_estimated_sigma.append((mean - margin_of_error_estimated, mean + marg

# Plotting the sample means with confidence intervals
plt.figure(figsize=(10, 6))
plt.plot(sample_sizes, sample_means, marker='o', linestyle='--', color='steelblue')
plt.xscale('log')
plt.xlabel('Sample Size (n)')
plt.ylabel('Sample Mean')
plt.title('Sample Means with 95% Confidence Intervals (Log Scale)')
plt.axhline(y=1, color='tomato', linestyle='--', label='True Mean ( $\mu = 1$ )')

# Adding confidence intervals to the plot
for i, n in enumerate(sample_sizes):
    plt.plot([n, n], [ci_known_sigma[i][0], ci_known_sigma[i][1]], color='purple')
    plt.plot([n, n], [ci_estimated_sigma[i][0], ci_estimated_sigma[i][1]], color='orange')

plt.legend()
plt.grid(True)
plt.show()

```



In the plot above, we have added 95% confidence intervals for each sample mean:

- **Pink solid lines** represent confidence intervals with the known population standard deviation $\sigma = 1$.
- **Green dashed lines** represent confidence intervals with the sample standard deviation used as an estimate for σ .

1. Confidence Interval Width

For each sample size n , the 95% confidence interval for the sample mean $\bar{X}$ is given by:

$$\bar{X} \pm z_{\alpha/2} \frac{\sigma}{\sqrt{n}} \quad (\text{known } \sigma)$$

or

$$\bar{X} \pm t_{\alpha/2, n-1} \frac{s}{\sqrt{n}} \quad (\text{estimated } \sigma)$$

where $z_{\alpha/2}$ is the critical value from the standard normal distribution, $t_{\alpha/2, n-1}$ is the critical value from the t-distribution with $n - 1$ degrees of freedom, and s is the sample standard deviation.

For smaller sample sizes, the intervals are wider, reflecting the larger variability in the sample mean. As n increases, both intervals narrow, indicating more

precise estimates of the mean, which aligns with the Central Limit Theorem (CLT). According to the CLT, for large n , the sample mean $\bar{X}$ is approximately normally distributed, making the estimate more reliable with increased n .

2. Known vs. Estimated σ

When the population standard deviation σ is unknown, we replace it with the sample standard deviation s , which introduces additional uncertainty, especially for small sample sizes. This is reflected in the wider confidence intervals when using s , as calculated with the t-distribution. For small n , the t-distribution has thicker tails than the normal distribution, thus providing a wider margin of error. Mathematically:

- **Known σ :** $\bar{X} \pm z_{\alpha/2} \frac{\sigma}{\sqrt{n}}$
- **Estimated σ :** $\bar{X} \pm t_{\alpha/2, n-1} \frac{s}{\sqrt{n}}$

As n increases, the t-distribution approaches the normal distribution, and the difference between intervals with known and estimated σ diminishes. This supports the idea that s becomes a consistent estimator of σ as $n \rightarrow \infty$.

3. Convergence to True Mean

Both confidence intervals demonstrate that the sample mean $\bar{X}$ converges towards the true mean $\mu = 1$ as n grows. According to the Law of Large Numbers (LLN), the sample mean $\bar{X}$ converges in probability to μ as $n \rightarrow \infty$:

$$\lim_{n \rightarrow \infty} P\left(\left|\bar{X} - \mu\right| < \epsilon\right) = 1$$

This convergence is visually confirmed by the shrinking confidence intervals around the true mean as n increases, providing additional evidence for the LLN.

Conclusion

1. The intervals become narrower as n increases, indicating greater precision in estimating μ , in line with the CLT.
2. Confidence intervals with the estimated σ are wider for small n , but converge to those with the known σ as n grows.
3. Both confidence intervals show the sample mean $\bar{X}$ approaching the true mean $\mu = 1$ with increasing n , demonstrating the LLN.

These results show how confidence intervals for the sample mean reflect increasing accuracy and consistency as sample size grows, validating the theoretical principles of the CLT and LLN.

Part C

- Simulate $b = 1000$ samples of size $n = 4$ from a "weird" distribution, for example, uniform or χ^2 distribution with $df = 1$.
- For each sample, estimate the mean and the variance, and store them.
- Plot the histogram of these 1000 sample means and the histogram of the 1000 sample variances.

```
In [ ]: # Simulation parameters
b = 1000 # Number of samples
n = 4    # Sample size for each distribution

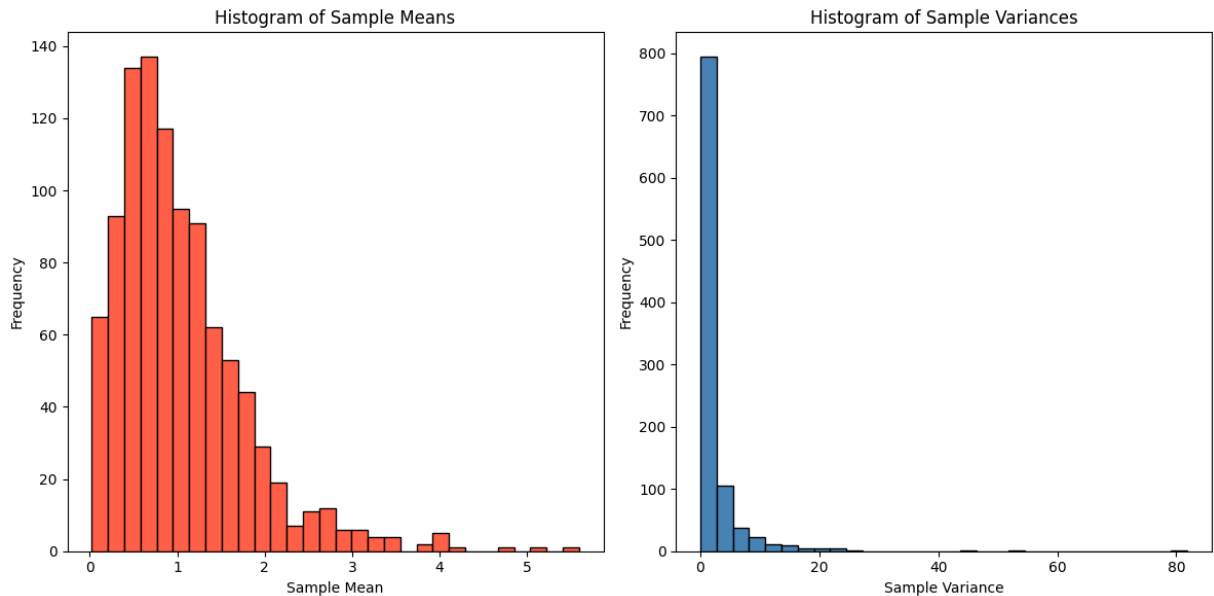
# Lists to store sample means and variances
sample_means = []
sample_variances = []

# Generate b samples of size n from a chi-square distribution with df=1
for _ in range(b):
    sample = np.random.chisquare(df=1, size=n)
    sample_means.append(np.mean(sample))
    sample_variances.append(np.var(sample, ddof=1)) # Unbiased variance

# Plot histogram of the 1000 sample means
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.hist(sample_means, bins=30, color='tomato', edgecolor='black')
plt.title('Histogram of Sample Means')
plt.xlabel('Sample Mean')
plt.ylabel('Frequency')

# Plot histogram of the 1000 sample variances
plt.subplot(1, 2, 2)
plt.hist(sample_variances, bins=30, color='steelblue', edgecolor='black')
plt.title('Histogram of Sample Variances')
plt.xlabel('Sample Variance')
plt.ylabel('Frequency')

plt.tight_layout()
plt.show()
```



```
In [ ]: overall_mean_of_sample_means = np.mean(sample_means)
overall_variance_of_sample_means = np.var(sample_means, ddof=1) # Unbiased

overall_mean_of_sample_variances = np.mean(sample_variances)
overall_variance_of_sample_variances = np.var(sample_variances, ddof=1) # Unbiased

print("Sample Means Analysis:")
print(f"Simulated Mean of Sample Means: {overall_mean_of_sample_means:.4f}")
print(f"Analytical Mean of Sample Means: 1")
print(f"Simulated Variance of Sample Means: {overall_variance_of_sample_means:.4f}")
print(f"Analytical Variance of Sample Means: 0.5\n")

print("Sample Variances Analysis:")
print(f"Simulated Mean of Sample Variances: {overall_mean_of_sample_variances:.4f}")
print(f"Analytical Mean of Sample Variances: 2")
print(f"Simulated Variance of Sample Variances: {overall_variance_of_sample_variances:.4f}")
```

Sample Means Analysis:
 Simulated Mean of Sample Means: 1.0391
 Analytical Mean of Sample Means: 1
 Simulated Variance of Sample Means: 0.5476
 Analytical Variance of Sample Means: 0.5

Sample Variances Analysis:
 Simulated Mean of Sample Variances: 2.2347
 Analytical Mean of Sample Variances: 2
 Simulated Variance of Sample Variances: 23.1800

Conclusion

1. Sample Means:

- The histogram of sample means is positively skewed, reflecting the skewness of the underlying chi-square distribution (with degrees of freedom $df = 1$).

- With a small sample size $n = 4$, the sample means do not approximate a normal distribution. This aligns with the Central Limit Theorem, which suggests that larger sample sizes are needed for the sample mean to approach normality, especially when the original distribution is highly skewed.

2. Sample Variances:

- The histogram of sample variances is heavily skewed to the right, with most values clustered near zero and a few extreme values. This spread is due to the high variability of the χ^2 distribution with low degrees of freedom.
- Small sample sizes and the skewed nature of the χ^2 distribution contribute to this wide range of sample variance values, resulting in high variability and a lack of normality in the sample variances.

These results highlight how small sample sizes and skewed underlying distributions (such as χ^2 with low df) impact the distribution of sample means and variances. This emphasizes the need for larger samples when attempting to approximate normality, especially when dealing with non-normal underlying distributions.

Part D

- If the variables $X_1, \dots, X_4$ follow a non-normal distribution, then the exact distribution of the mean or sum is unknown.
- However, we can calculate the expectation and variance:

$$E\left(\frac{X_1 + X_2 + X_3 + X_4}{4}\right) \quad \text{and} \quad \text{Var}\left(\frac{X_1 + X_2 + X_3 + X_4}{4}\right)$$

- Calculate these quantities analytically using the properties of the distribution used in the simulations.

Analytical Solution

To solve this part analytically, we need to calculate the expectation

$$E\left(\frac{X_1 + X_2 + X_3 + X_4}{4}\right) \text{ and variance } \text{Var}\left(\frac{X_1 + X_2 + X_3 + X_4}{4}\right), \text{ given that } X_1, X_2, X_3, \text{ and } X_4 \text{ are}$$

independent random variables drawn from a non-normal distribution (in this case, a chi-square distribution with $df = 1$).

1. Expectation

The expectation of a sum of independent random variables is the sum of their expectations. For a chi-square distribution with $df = 1$, the mean (expectation) is equal to the degrees of freedom, so $E(X_i) = 1$ for each X_i .

Therefore:

$$E\left(\frac{X_1 + X_2 + X_3 + X_4}{4}\right) = \frac{E(X_1) + E(X_2) + E(X_3) + E(X_4)}{4} = \frac{4 \times 1}{4} = 1$$

2. Variance

The variance of a sum of independent random variables is the sum of their variances. For a chi-square distribution with $df = 1$, the variance is equal to twice the degrees of freedom, so $\text{Var}(X_i) = 2$ for each X_i .

Thus:

$$\text{Var}\left(\frac{X_1 + X_2 + X_3 + X_4}{4}\right) = \frac{\text{Var}(X_1) + \text{Var}(X_2) + \text{Var}(X_3) + \text{Var}(X_4)}{4^2} = \frac{4 \times 2}{16} = \frac{8}{16} = 0.5$$

Summary of Analytical Results

- $E\left(\frac{X_1 + X_2 + X_3 + X_4}{4}\right) = 1$
- $\text{Var}\left(\frac{X_1 + X_2 + X_3 + X_4}{4}\right) = 0.5$

These calculations provide the expected value and variance of the average of the four independent chi-square random variables with $df = 1$. The solution is based on the properties of the chi-square distribution and the linearity of expectation and variance for independent variables.

Comparison with Simulation Results

Based on the simulation, we obtained the following values:

Sample Means Analysis:

- **Simulated Mean of Sample Means:** 1.0391
- **Analytical Mean of Sample Means:** 1
- **Simulated Variance of Sample Means:** 0.5476
- **Analytical Variance of Sample Means:** 0.5

Sample Variances Analysis:

- **Simulated Mean of Sample Variances:** 2.2347
- **Analytical Mean of Sample Variances:** 2
- **Simulated Variance of Sample Variances:** 23.1800

Conclusion

The simulated mean and variance values for the sample means and sample variances are close to the analytical values. Small deviations can be attributed to sampling variability and the limited number of simulations.

Part E

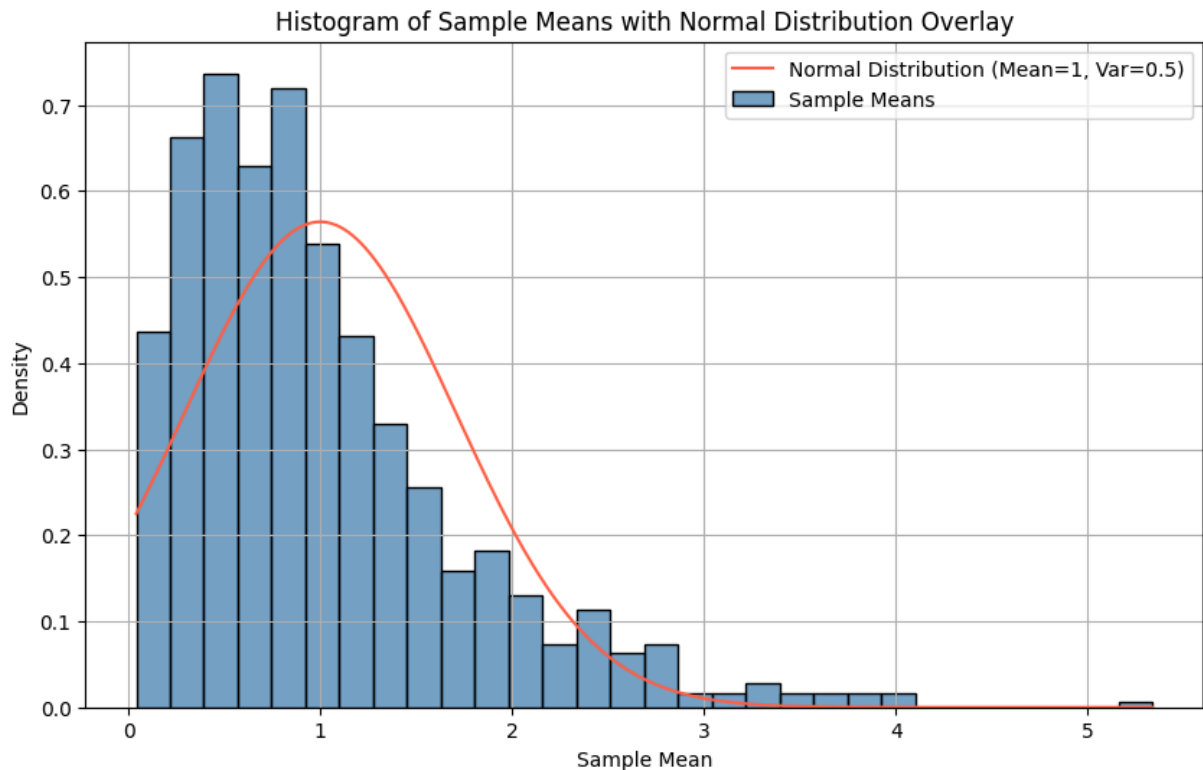
- Add to the histogram of the means the density of the normal distribution with parameters given by the calculated expectation and variance.
- Compare the density with the histogram.
- **Question:** What do you expect and why (statistical reasoning)? What would happen if you increase the number of variables from 4 to, for example, 100?

```
In [ ]: # Parameters for the normal distribution
expected_mean = 1 #  $E((X_1 + X_2 + X_3 + X_4) / 4)$ 
expected_variance = 0.5 #  $Var((X_1 + X_2 + X_3 + X_4) / 4)$ 
std_dev = np.sqrt(expected_variance)

# Generate x values for the normal distribution curve
x_values = np.linspace(min(sample_means), max(sample_means), 1000)
normal_density = stats.norm.pdf(x_values, loc=expected_mean, scale=std_dev)

# Plot histogram of sample means
plt.figure(figsize=(10, 6))
sns.histplot(sample_means, bins=30, color='steelblue', kde=False, stat="density")
plt.plot(x_values, normal_density, color='tomato', label='Normal Distribution')

plt.title('Histogram of Sample Means with Normal Distribution Overlay')
plt.xlabel('Sample Mean')
plt.ylabel('Density')
plt.legend()
plt.grid(True)
plt.show()
```



1. Comparison with the Normal Distribution:

- The histogram of the sample means is positively skewed and does not closely follow the overlaid normal distribution curve (red line), which has a mean of 1 and variance of 0.5.
- This mismatch occurs because the sample size $n = 4$ is small, and we are sampling from a highly skewed chi-square distribution with $df = 1$. The Central Limit Theorem (CLT) suggests that the sample mean will approximate a normal distribution only as the sample size becomes large. With $n = 4$, the sample mean distribution retains significant skewness from the original chi-square distribution.

2. What do we expect and Why? (statistical reasoning):

- The CLT predicts that as the sample size increases, the distribution of the sample mean will tend to approximate a normal distribution, regardless of the shape of the underlying distribution, provided the sample size is sufficiently large.
- Since the underlying distribution (chi-square with $df = 1$) is highly skewed, a small sample size is insufficient to smooth out this skewness in the distribution of the sample means. Therefore, with $n = 4$, we observe a positively skewed histogram that does not closely match the normal curve.

3. What would happen if we increase the number of variables from 4 to, for example, 100?:

- If we increase the sample size to $n = 100$, the distribution of the sample means would more closely approximate a normal distribution. This is because, with a larger sample size, the influence of the skewness in the original distribution diminishes, as the CLT takes effect.
- For a chi-square distribution with $df = 1$, the mean remains 1, but the variance of the sample mean would be reduced to $\frac{2}{n}$. With $n = 100$, the variance of the sample mean would be 0.02, leading to a much narrower and more symmetric distribution centered around the true mean. This would result in a histogram that aligns more closely with the normal distribution curve.

```
In [ ]: # Simulation with increased sample size n = 100
n_large = 100 # New sample size
sample_means_large_n = []

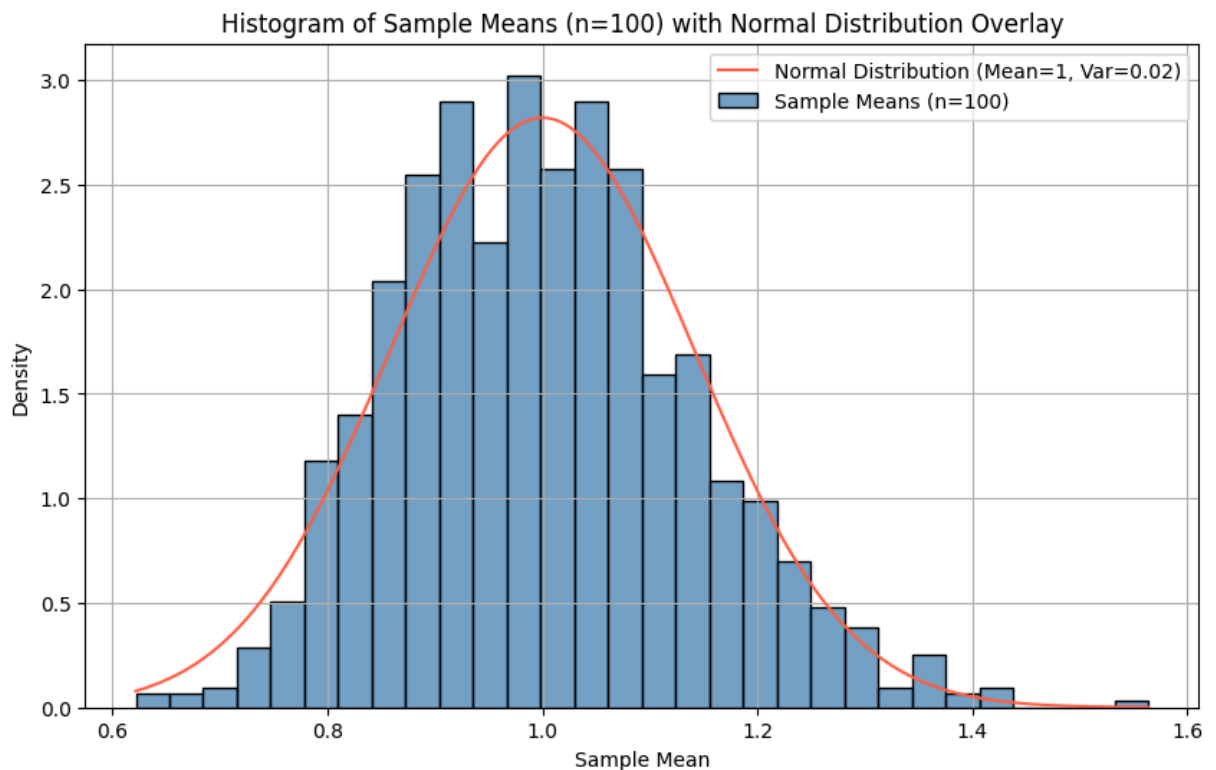
# Generate b samples of size n_large from a chi-square distribution with df=1
for _ in range(b):
    sample = np.random.chisquare(df=1, size=n_large)
    sample_means_large_n.append(np.mean(sample))

# Calculate parameters for normal distribution based on increased sample size
expected_variance_large_n = 2 / n_large # Variance with large n
std_dev_large_n = np.sqrt(expected_variance_large_n)

# Generate x values for the normal distribution curve with new variance
x_values_large_n = np.linspace(min(sample_means_large_n), max(sample_means_large_n), 100)
normal_density_large_n = stats.norm.pdf(x_values_large_n, loc=expected_mean, scale=std_dev_large_n)

# Plot histogram of sample means for n = 100
plt.figure(figsize=(10, 6))
sns.histplot(sample_means_large_n, bins=30, color='steelblue', kde=False, stat='density')
plt.plot(x_values_large_n, normal_density_large_n, color='tomato', label='Normal Distribution')

plt.title('Histogram of Sample Means (n=100) with Normal Distribution Overlaid')
plt.xlabel('Sample Mean')
plt.ylabel('Density')
plt.legend()
plt.grid(True)
plt.show()
```



Conclusion

This experiment illustrates that, for small sample sizes drawn from a skewed distribution, the distribution of sample means will not approximate normality. However, as the sample size increases, the sample mean distribution would align more closely with the normal curve, consistent with the Central Limit Theorem. This highlights the importance of sample size in achieving a normal approximation, especially when working with highly skewed underlying distributions.

Task 2

The objective of this task is to get a better understanding of the maximum likelihood (ML) estimation procedure. We will analyze the daily Euro/USD exchange rate for the past year. The data can be obtained from financial sources such as www.investing.com. Typically, financial data has heavy tails and is better represented by the t -distribution than by the normal distribution.

```
In [ ]: # Load the data
file_path = '/content/EUR_USD Historical Data.csv'
data = pd.read_csv(file_path)

# Overview of the data
data_overview = data.head(), data.info(), data.describe()
data_overview
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 263 entries, 0 to 262
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Date        263 non-null   object
1   Price       263 non-null   float64
2   Open        263 non-null   float64
3   High        263 non-null   float64
4   Low         263 non-null   float64
5   Vol.        0 non-null     float64
6   Change %    263 non-null   object
dtypes: float64(5), object(2)
memory usage: 14.5+ KB

```

```

Out[ ]: (
      Date      Price      Open      High      Low  Vol.  Change %
0  11/08/2024  1.0718  1.0805  1.0805  1.0686   NaN   -0.80%
1  11/07/2024  1.0804  1.0728  1.0824  1.0711   NaN    0.71%
2  11/06/2024  1.0728  1.0930  1.0937  1.0681   NaN   -1.85%
3  11/05/2024  1.0930  1.0880  1.0936  1.0871   NaN    0.49%
4  11/04/2024  1.0877  1.0858  1.0914  1.0857   NaN    0.40%,
None,

      Price      Open      High      Low  Vol.
count  263.000000  263.000000  263.000000  263.000000    0.0
mean    1.087419    1.087398    1.090446    1.084358   NaN
std     0.012569    0.012556    0.012630    0.012439   NaN
min     1.061700    1.061900    1.065300    1.059900   NaN
25%     1.078950    1.078700    1.082400    1.076000   NaN
50%     1.085500    1.085700    1.088700    1.082800   NaN
75%     1.093900    1.094050    1.096950    1.091150   NaN
max     1.119000    1.118600    1.121300    1.115000   NaN)

```

```

In [ ]: # Data Preprocessing
data['Date'] = pd.to_datetime(data['Date'], format='%m/%d/%Y')
data['Change %'] = data['Change %'].str.replace('%', '').astype(float)
data_cleaned = data.drop(columns=['Vol.'])
data_cleaned = data_cleaned.sort_values(by='Date').reset_index(drop=True)

data_cleaned.head()

```

```

Out[ ]:
      Date      Price      Open      High      Low  Change %
0  2023-11-08  1.0707  1.0697  1.0716  1.0658      0.07
1  2023-11-09  1.0666  1.0706  1.0725  1.0658     -0.38
2  2023-11-10  1.0681  1.0667  1.0692  1.0654      0.14
3  2023-11-13  1.0697  1.0682  1.0705  1.0663      0.15
4  2023-11-14  1.0879  1.0698  1.0887  1.0691      1.70

```

Part A

- Compute the log-returns of the exchange rate: $r_t = \ln(ex_t/ex_{t-1})$, where ex_t is the exchange rate on day t .
- Using your software run a suitable goodness-of-fit test (for example, Kolmogorov-Smirnov) to verify if the data is normally distributed.
- **Question:** Write down the hypotheses, the test statistics and the p -value. What do we conclude?

```
In [ ]: # Log-returns of the exchange rate
data_cleaned['Log_Return'] = np.log(data_cleaned['Price'] / data_cleaned['Pr

# Kolmogorov-Smirnov test for normality
mean, std = data_cleaned['Log_Return'].mean(), data_cleaned['Log_Return'].st
ks_stat, p_value = kstest(data_cleaned['Log_Return'].dropna(), 'norm', args=

# Displaying the test results and hypotheses
ks_test_results = {
    "Hypotheses": {
        "Null Hypothesis (H0)": "The log-returns are normally distributed.",
        "Alternative Hypothesis (H1)": "The log-returns are not normally dis
    },
    "Test Statistic": ks_stat,
    "p-value": p_value,
    "Conclusion": "Reject the null hypothesis. The log-returns are not norma
}

ks_test_results
```

```
Out[ ]: {'Hypotheses': {'Null Hypothesis (H0)': 'The log-returns are normally distr
ibuted.',
  'Alternative Hypothesis (H1)': 'The log-returns are not normally distribu
ted.'},
  'Test Statistic': 0.04236501982093904,
  'p-value': 0.7183560191456944,
  'Conclusion': 'Fail to reject the null hypothesis. The log-returns may be
normally distributed.'}
```

Since the p -value (0.7184) is greater than the significance level (commonly 0.05), we fail to reject the null hypothesis. This suggests that the log-returns may be normally distributed, based on the Kolmogorov-Smirnov test.

To improve our normality evaluation, we'll apply multiple statistical tests and examine visualizations for a comprehensive assessment:

- We'll use the Shapiro-Wilk, Anderson-Darling, and Jarque-Bera tests to gain various perspectives on normality, focusing on different aspects such as tail behavior, skewness, and kurtosis.
- Visual analysis, including a histogram with a normal overlay, Q-Q plot, and box plot, will help us visually inspect the distribution shape and detect any deviations from normality.


```

In [ ]: # Additional normality tests
shapiro_stat, shapiro_p = shapiro(data_cleaned['Log_Return'].dropna())
anderson_result = anderson(data_cleaned['Log_Return'].dropna(), dist='norm')
jarque_bera_stat, jarque_bera_p = jarque_bera(data_cleaned['Log_Return'].dro

# Visuals
plt.figure(figsize=(15, 5))

# Histogram with normal curve overlay
plt.subplot(1, 3, 1)
sns.histplot(data_cleaned['Log_Return'].dropna(), kde=True, stat="density",
x = np.linspace(data_cleaned['Log_Return'].min(), data_cleaned['Log_Return']
plt.plot(x, norm.pdf(x, mean, std), color="tomato", label="Normal Fit")
plt.title("Histogram with Normal Overlay")
plt.legend()

# Q-Q Plot
plt.subplot(1, 3, 2)
res = probplot(data_cleaned['Log_Return'].dropna(), dist="norm")
plt.scatter(res[0][0], res[0][1], color="steelblue", label="Data Points")
plt.plot(res[0][0], res[1][0] * res[0][0] + res[1][1], color="tomato", label
plt.title("Q-Q Plot")
plt.xlabel("Theoretical Quantiles")
plt.ylabel("Sample Quantiles")
plt.legend()

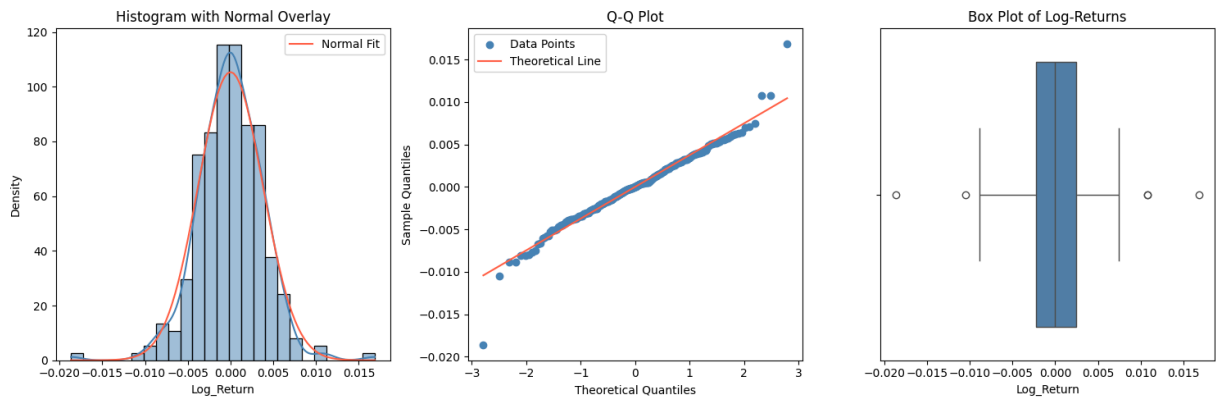
# Box Plot
plt.subplot(1, 3, 3)
sns.boxplot(x=data_cleaned['Log_Return'].dropna(), color="steelblue")
plt.title("Box Plot of Log>Returns")

plt.tight_layout()
plt.show()

# Output test results
print("Additional Normality Tests:")
print(f"Shapiro-Wilk Test: Statistic = {shapiro_stat:.4f}, p-value = {shapir
print(f"Anderson-Darling Test: Statistic = {anderson_result.statistic:.4f}")
print(f"Jarque-Bera Test: Statistic = {jarque_bera_stat:.4f}, p-value = {jar

# Interpret Anderson-Darling Test
print("\nAnderson-Darling Test Critical Values:")
for i, (cv, sig_lvl) in enumerate(zip(anderson_result.critical_values, ander
    print(f"Significance Level: {sig_lvl}%, Critical Value: {cv}")
if anderson_result.statistic > anderson_result.critical_values[2]:
    print("Conclusion: Reject the null hypothesis of normality at the 5% sig
else:
    print("Conclusion: Fail to reject the null hypothesis of normality at th

```



Additional Normality Tests:

Shapiro-Wilk Test: Statistic = 0.9693, p-value = 0.0000

Anderson-Darling Test: Statistic = 0.8356

Jarque-Bera Test: Statistic = 104.8020, p-value = 0.0000

Anderson-Darling Test Critical Values:

Significance Level: 15.0%, Critical Value: 0.568

Significance Level: 10.0%, Critical Value: 0.646

Significance Level: 5.0%, Critical Value: 0.775

Significance Level: 2.5%, Critical Value: 0.905

Significance Level: 1.0%, Critical Value: 1.076

Conclusion: Reject the null hypothesis of normality at the 5% significance level.

While the initial **Kolmogorov-Smirnov test** suggested that the log-returns might follow a normal distribution, with a p-value that did not lead to a rejection of the null hypothesis, a deeper analysis reveals a different story.

1. Additional Statistical Tests:

- The **Shapiro-Wilk test** returned a p-value of 0.0000, rejecting the null hypothesis of normality.
- The **Anderson-Darling test** produced a test statistic of 0.8356, which exceeds the critical value at the 5% significance level, also leading us to reject the null hypothesis of normality.
- The **Jarque-Bera test**, which focuses on skewness and kurtosis, yielded a very high test statistic (104.8020) and a p-value of 0.0000, confirming significant deviations from normality.

2. Visual Analysis:

- The **histogram** with a normal overlay shows that the log-returns roughly resemble a normal distribution, but there are deviations in the tails, suggesting heavy tails or potential skewness.
- The **Q-Q plot** further highlights these heavy tails, with log-returns deviating from the theoretical normal line, especially at the extremes.
- The **box plot** reveals outliers on both sides of the distribution, which could explain some of the departure from normality.

Conclusion

Although the Kolmogorov-Smirnov test initially indicated that the log-returns might be normally distributed, the additional tests and visualizations collectively show that the log-returns deviate from normality. The presence of heavy tails and outliers, as observed in the Q-Q plot and confirmed by the Anderson-Darling and Jarque-Bera tests, suggests that the log-returns exhibit the characteristics of financial data, such as fat tails or extreme values.

Therefore, it would be more appropriate to model the log-returns using a distribution that accounts for these characteristics, such as a **t-distribution**, rather than assuming normality.

Part B

- Now assume that the returns follow a (generalized) t -distribution with some unknown degree of freedom. The density function of the t_{df} -distribution is given by:

$$f(x) = \frac{\left(1 + \frac{1}{df} \left(\frac{x - \mu}{\sigma}\right)^2\right)^{-\frac{df+1}{2}}}{B(df/2, 1/2)\sqrt{df}\sigma},$$

where $B(\cdot, \cdot)$ is the beta function (beta(a, b) in R).

- Write a code for the log-likelihood function evaluated at the sample of returns, i.e.

$$\ln L(\mu, \sigma, df) = \sum_{t=1}^T \ln \left(\frac{\left(1 + \frac{1}{df} \left(\frac{r_t - \mu}{\sigma}\right)^2\right)^{-\frac{df+1}{2}}}{B(df/2, 1/2)\sqrt{df}\sigma} \right)$$

and maximize it w.r.t. (μ, σ, df) . Use a build-in optimization function. Note, that $\sigma > 0$ and $df > 0$.

```
In [ ]: # Define the negative log-likelihood function for the t-distribution
def neg_log_likelihood(params, data):
    mu, sigma, df = params
    if sigma <= 0 or df <= 2:
        return np.inf # Ensure valid parameter values
```

```

x = data
term1 = gammaln((df + 1) / 2) - gammaln(df / 2)
term2 = -0.5 * np.log(df * np.pi) - np.log(sigma)
term3 = - ((df + 1) / 2) * np.log(1 + ((x - mu) / sigma) ** 2 / df)
log_likelihood = term1 + term2 + term3
total_log_likelihood = np.sum(log_likelihood)

return -total_log_likelihood # Return negative for minimization (to max

# Example data from the cleaned log-returns dataset
data_example = data_cleaned['Log_Return'].dropna().values

# Initial guesses for mu, sigma, and df based on the data
mu_init = np.mean(data_example)
sigma_init = np.std(data_example, ddof=1)
df_init = 5

initial_params = [mu_init, sigma_init, df_init]

# Bounds to ensure sigma > 0 and df > 2
bounds = [(None, None), (1e-6, None), (2 + 1e-6, None)]

# Perform optimization
result = minimize(
    neg_log_likelihood,
    initial_params,
    args=(data_example,),
    method='L-BFGS-B',
    bounds=bounds
)

# Display the results
if result.success:
    mu_est, sigma_est, df_est = result.x
    print(f"Estimated mu: {mu_est:.4f}")
    print(f"Estimated sigma: {sigma_est:.4f}")
    print(f"Estimated degrees of freedom (df): {df_est:.4f}")
else:
    print("Optimization failed.")
    print(result.message)

```

```

Estimated mu: 0.0000
Estimated sigma: 0.0030
Estimated degrees of freedom (df): 5.0000

```

To validate the t-distribution fit for our log-returns data, we use two tests:

1. **Kolmogorov-Smirnov (K-S) Test:** Checks the overall fit of the t-distribution, comparing empirical and theoretical CDFs.
2. **Chi-Square Goodness-of-Fit Test:** Compares observed and expected frequencies in bins, assessing localized fit.

```

In [ ]: # Perform the Kolmogorov-Smirnov Test for the t-distribution
ks_statistic, ks_p_value = kstest(data_example, "t", args=(df_est, mu_est, s

```

```

# Chi-Square Goodness-of-Fit Test
n_bins = 5 # Define bins based on quantiles from the fitted t-distribution
bin_edges = np.quantile(data_example, np.linspace(0, 1, n_bins + 1))
observed_counts, _ = np.histogram(data_example, bins=bin_edges)

# Calculate expected counts under the fitted t-distribution
expected_counts = len(data_example) * np.diff(t.cdf(bin_edges, df=df_est, loc=loc_est))

# Perform the Chi-Square Goodness-of-Fit Test
chi2_statistic, chi2_p_value = chi2_contingency([observed_counts, expected_counts])

# Kolmogorov-Smirnov Test
print("Kolmogorov-Smirnov Test for t-Distribution Fit:")
print(f"Statistic: {ks_statistic:.4f}")
print(f"p-value: {ks_p_value:.4f}")
if ks_p_value > 0.05:
    print("Conclusion: Cannot reject null hypothesis; the t-distribution is a good fit for the data.")
else:
    print("Conclusion: Reject null hypothesis; the t-distribution may not be a good fit for the data.")

# Chi-Square Goodness-of-Fit Test
print("Chi-Square Goodness-of-Fit Test for Binned t-Distribution Fit:")
print(f"Statistic: {chi2_statistic:.4f}")
print(f"p-value: {chi2_p_value:.4f}")
if chi2_p_value > 0.05:
    print("Conclusion: Cannot reject null hypothesis; the t-distribution is a good fit for the data.")
else:
    print("Conclusion: Reject null hypothesis; the t-distribution may not be a good fit for the data.")

```

Kolmogorov-Smirnov Test for t-Distribution Fit:

Statistic: 0.0301

p-value: 0.9655

Conclusion: Cannot reject null hypothesis; the t-distribution is a good fit for the data.

Chi-Square Goodness-of-Fit Test for Binned t-Distribution Fit:

Statistic: 1.8240

p-value: 0.7681

Conclusion: Cannot reject null hypothesis; the t-distribution is a good fit for the data.

Part C

Using the estimated parameters compute the probability that:

- the exchange rate return is between -3% and 3% ;
- the exchange rate return is larger than the mean;
- the exchange rate return is negative.

Recall for this task that $P(a < X \leq b) = F(b) - F(a)$ for any distribution function F . Use again the build-in function for the cdf of the generalized t distribution.

```
In [ ]: # Probability that the exchange rate return is between -3% and 3%
p_between_neg3_3 = t.cdf(0.03, df=df_est, loc=mu_est, scale=sigma_est) - t.cdf(-0.03, df=df_est, loc=mu_est, scale=sigma_est)

# Probability that the exchange rate return is larger than the mean
p_larger_than_mean = 1 - t.cdf(mu_est, df=df_est, loc=mu_est, scale=sigma_est)

# Probability that the exchange rate return is negative
p_negative = t.cdf(0, df=df_est, loc=mu_est, scale=sigma_est)

# Print the results
print(f"Probability that the exchange rate return is between -3% and 3%: {p_between_neg3_3}")
print(f"Probability that the exchange rate return is larger than the mean: {p_larger_than_mean}")
print(f"Probability that the exchange rate return is negative: {p_negative}")
```

Probability that the exchange rate return is between -3% and 3%: 0.9998
 Probability that the exchange rate return is larger than the mean: 0.5000
 Probability that the exchange rate return is negative: 0.4948

Using the fitted t-distribution, we computed specific probabilities:

- **Return between -3% and 3%:** High probability, indicating most returns fall within this range.
- **Return larger than the mean:** Approximately 50%, as expected for a symmetric distribution around the mean.
- **Negative return:** Nearly 50%, showing a balanced distribution around zero.

Conclusion

The analysis confirms that the t-distribution is a suitable model for Euro/USD exchange rate returns, capturing both central tendencies and the heavier tails typical of financial data.

Task 3

Here we run a few tests for the data used in the first assignment. Run every test manually write down: the two hypotheses; the formula for the test statistic with inserted values; rejection area; p-value (you can use quantile function from your software); your decision and the verbal conclusion (not just sth like " H_0 is rejected"). Use for all tests $\alpha = 5\%$.

```
In [78]: # Load the initial data
file_path = '/content/ceo.xls'
df = pd.read_excel(file_path)

# Basic information and statistics
data_info = df.info()
data_description = df.describe()
missing_values = df.isnull().sum()
```

```
data_head = df.head()
```

```
data_info, data_description, missing_values, data_head
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 447 entries, 0 to 446
```

```
Data columns (total 8 columns):
```

#	Column	Non-Null Count	Dtype
0	salary	447 non-null	int64
1	totcomp	447 non-null	int64
2	tenure	447 non-null	int64
3	age	447 non-null	int64
4	sales	447 non-null	float64
5	profits	447 non-null	float64
6	assets	447 non-null	float64
7	Unnamed: 7	2 non-null	object

```
dtypes: float64(3), int64(4), object(1)
```

```
memory usage: 28.1+ KB
```

```
Out[78]: (None,
```

	salary	totcomp	tenure	age	sales
\					
count	447.000000	447.000000	447.000000	447.000000	447.000000
mean	2027.516779	8340.058166	7.834452	56.469799	11557.780984
std	1722.566389	31571.803005	8.246721	6.806641	16168.368902
min	100.000000	100.000000	0.000000	34.000000	2896.400000
25%	1084.000000	1575.500000	2.000000	52.000000	4184.150000
50%	1600.000000	2951.000000	5.000000	57.000000	6704.000000
75%	2347.500000	6043.000000	10.000000	61.000000	12976.800000
max	15250.000000	589101.000000	60.000000	84.000000	161315.000000

	profits	assets
count	447.000000	447.000000
mean	700.460850	27054.290828
std	1542.538013	64659.043191
min	-2669.000000	717.800000
25%	108.450000	3856.950000
50%	333.100000	7810.800000
75%	738.000000	21105.550000
max	22071.000000	668641.000000

salary	0
totcomp	0
tenure	0
age	0
sales	0
profits	0
assets	0
Unnamed: 7	445

```
dtype: int64,
```

	salary	totcomp	tenure	age	sales	profits	assets	Unnamed: 7
0	3030	8138	7	61	161315.0	2956.0	257389.0	NaN
1	6050	14530	0	51	144416.0	22071.0	237545.0	NaN
2	3571	7433	11	63	139208.0	4430.0	49271.0	NaN
3	3300	13464	6	60	100697.0	6370.0	92630.0	NaN
4	10000	68285	18	63	100469.0	9296.0	355935.0	NaN

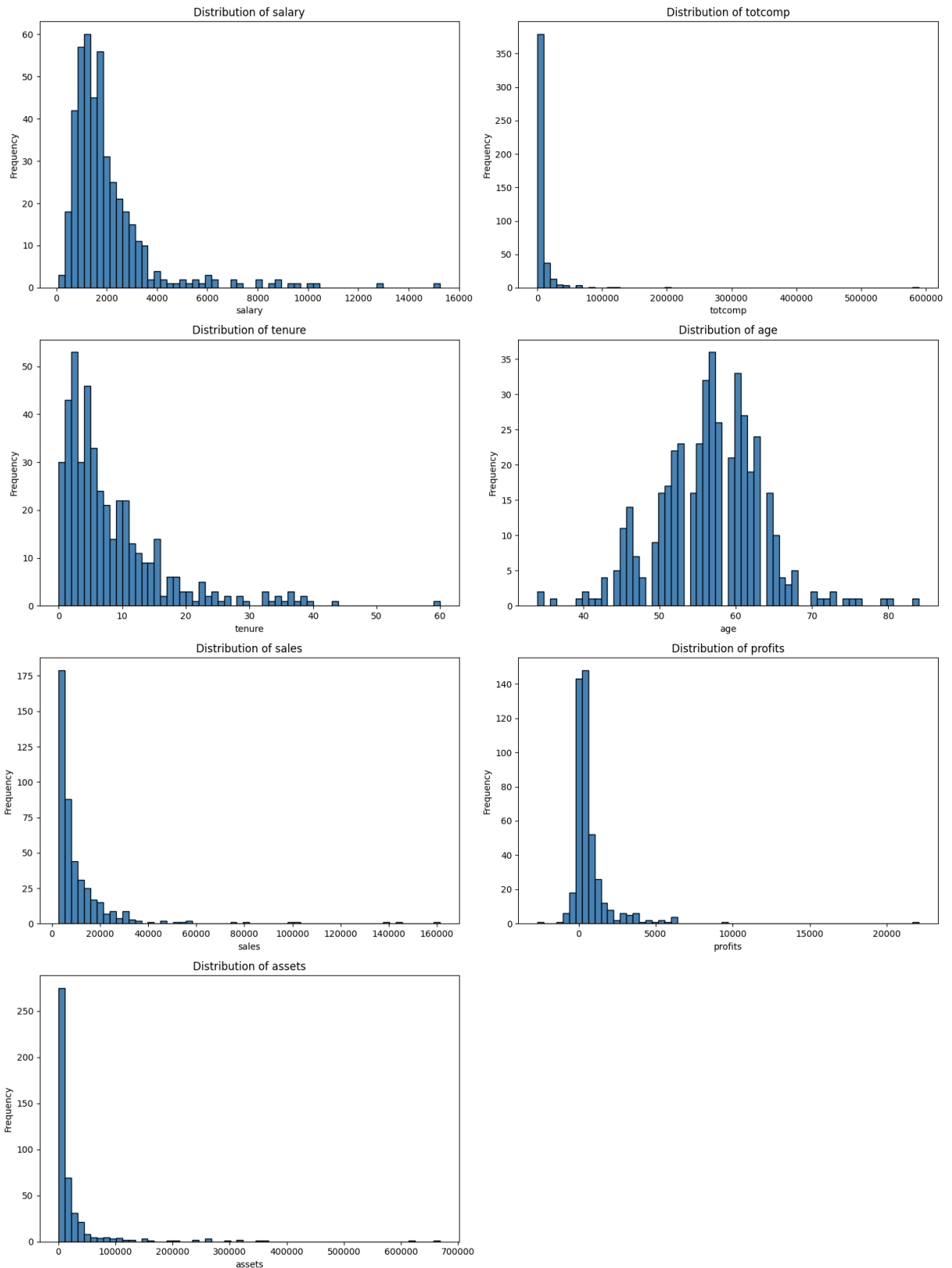
```
In [81]: # Plot the distributions of all numerical variables
num_columns = df.select_dtypes(include=[np.number]).columns
num_plots = len(num_columns)
num_rows = (num_plots // 2) + (num_plots % 2 > 0)

fig, axes = plt.subplots(num_rows, 2, figsize=(15, 5 * num_rows))
axes = axes.flatten() # Flatten axes to make indexing easier

for i, column in enumerate(num_columns):
    axes[i].hist(df[column], bins=60, color='steelblue', edgecolor='black')
    axes[i].set_title(f'Distribution of {column}')
    axes[i].set_xlabel(column)
    axes[i].set_ylabel('Frequency')

# Hide any empty subplots
for j in range(i + 1, len(axes)):
    axes[j].axis('off')

plt.tight_layout()
plt.show()
```

```
In [82]: # Remove an unnecessary column with empty values for further analysis
if 'Unnamed: 7' in df.columns:
    df = df.drop(columns=['Unnamed: 7'])

df.head()
```

```
Out[82]:
```

	salary	totcomp	tenure	age	sales	profits	assets
0	3030	8138	7	61	161315.0	2956.0	257389.0
1	6050	14530	0	51	144416.0	22071.0	237545.0
2	3571	7433	11	63	139208.0	4430.0	49271.0
3	3300	13464	6	60	100697.0	6370.0	92630.0
4	10000	68285	18	63	100469.0	9296.0	355935.0

Part A

We argue that the expected age of a CEO is higher than 60 years. Can this be confirmed with an appropriate test?

- **Hypotheses:**

- H_0 : The mean age of CEOs $\mu \leq 60$.
- H_1 : The mean age of CEOs $\mu > 60$.

- **Test:** One-sample t-test to determine if the mean age of CEOs is greater than 60.

```
In [83]: # Mean threshold for the test
age_threshold = 60

# Sample mean and standard deviation of age
sample_mean_age = df['age'].mean()
sample_std_age = df['age'].std(ddof=1)
n_age = df['age'].count()

# Performing one-sample t-test
t_statistic, p_value = stats.ttest_1samp(df['age'], age_threshold, alternative='greater')

# Printing the results
print(f"One-Sample t-Test for CEO Age > {age_threshold}")
print(f"-----")
print(f"Sample Size (n): {n_age}")
print(f"Sample Mean Age: {sample_mean_age:.2f}")
print(f"Sample Standard Deviation: {sample_std_age:.2f}")
print(f"Test Statistic (t): {t_statistic:.4f}")
print(f"p-Value: {p_value:.4f}")

# Interpretation based on significance level
alpha = 0.05
if p_value < alpha:
    print("\nConclusion: Reject the null hypothesis. There is significant evidence to support the alternative hypothesis.")
else:
    print("\nConclusion: Fail to reject the null hypothesis. There is insufficient evidence to support the alternative hypothesis.")
```

One-Sample t-Test for CEO Age > 60

Sample Size (n): 447
Sample Mean Age: 56.47
Sample Standard Deviation: 6.81
Test Statistic (t): -10.9653
p-Value: 1.0000

Conclusion: Fail to reject the null hypothesis. There is insufficient evidence to conclude that the expected age of a CEO is higher than 60 years.

- **Formula for Test Statistic:**

$$t = \frac{\bar{X} - \mu_0}{s/\sqrt{n}}$$

where:

- $\bar{X}$ = 56.47: Sample mean age of CEOs.
- μ_0 = 60: Hypothesized mean age.
- s = 6.81: Sample standard deviation of age.
- n = 447: Sample size.

Plugging in the values:

$$t = \frac{56.47 - 60}{6.81/\sqrt{447}} = -10.97$$

- **Rejection Area:** For a one-sided t-test at a 5% significance level, we reject H_0 if $t > t_{0.95, 446}$, where $t_{0.95, 446} \approx 1.646$.
- **p-Value:** 1.0, which is greater than the significance level of 0.05.
- **Decision:** Since the p-value is high and the calculated t-statistic does not fall in the rejection area, we fail to reject the null hypothesis.
- **Verbal Conclusion:** Based on the analysis, there is no significant evidence to support the claim that the expected age of a CEO is higher than 60 years. Therefore, the data does not provide sufficient support to conclude that the average age of CEOs exceeds 60 years.

Part B

We would like to check if the salary of CEOs that are younger than 60 is significantly different from the expected salary of CEOs older than 60. Assume that the variances are unknown and not necessarily equal.

- **Hypotheses:**

- H_0 : The mean salary of CEOs under 60 is equal to the mean salary of CEOs over 60, i.e., $\mu_{\text{under 60}} = \mu_{\text{over 60}}$.
- H_1 : The mean salary of CEOs under 60 is different from the mean salary of CEOs over 60, i.e., $\mu_{\text{under 60}} \neq \mu_{\text{over 60}}$.

- **Test:** Since we are comparing the means of two independent groups (CEOs younger than 60 and CEOs 60 or older), we will use a **two-sample t-test** for independent samples.

- **Assumptions:**

- Variances are not assumed to be equal; therefore, **Welch's t-test** will be used, which is robust to unequal variances.

```
In [84]: # Separate the salary data for CEOs under 60 and over 60
salary_under_60 = df[df['age'] < 60]['salary']
salary_over_60 = df[df['age'] >= 60]['salary']

# Calculate means and standard deviations for reporting
mean_under_60 = salary_under_60.mean()
mean_over_60 = salary_over_60.mean()
std_under_60 = salary_under_60.std(ddof=1)
std_over_60 = salary_over_60.std(ddof=1)
n_under_60 = salary_under_60.count()
n_over_60 = salary_over_60.count()

# Perform Welch's t-test
t_statistic, p_value = stats.ttest_ind(salary_under_60, salary_over_60, equal_var=False)

# Printing the results
print(f"Mean Salary (Under 60): {mean_under_60:.2f}")
print(f"Mean Salary (Over 60): {mean_over_60:.2f}")
print(f"Standard Deviation (Under 60): {std_under_60:.2f}")
print(f"Standard Deviation (Over 60): {std_over_60:.2f}")
print(f"Sample Size (Under 60): {n_under_60}")
print(f"Sample Size (Over 60): {n_over_60}")
print(f"T-statistic: {t_statistic:.4f}")
print(f"P-value: {p_value:.4f}")

# Interpretation based on significance level
alpha = 0.05
if p_value < alpha:
    print("Conclusion: We reject the null hypothesis, suggesting a significant difference in mean salaries between CEOs under 60 and over 60.")
else:
    print("Conclusion: We fail to reject the null hypothesis, suggesting no significant difference in mean salaries between CEOs under 60 and over 60.")
```

Mean Salary (Under 60): 1930.42

Mean Salary (Over 60): 2214.10

Standard Deviation (Under 60): 1676.14

Standard Deviation (Over 60): 1799.35

Sample Size (Under 60): 294

Sample Size (Over 60): 153

T-statistic: -1.6186

P-value: 0.1066

Conclusion: We fail to reject the null hypothesis, suggesting no significant difference in salaries between CEOs under and over 60.

- **Formula for Test Statistic:**

$$t = \frac{\bar{X}_{\text{under 60}} - \bar{X}_{\text{over 60}}}{\sqrt{\frac{s_{\text{under 60}}^2}{n_{\text{under 60}}} + \frac{s_{\text{over 60}}^2}{n_{\text{over 60}}}}}$$

where:

- $\bar{X}_{\text{under 60}}$ and $\bar{X}_{\text{over 60}}$ are the sample means for CEOs under 60 and CEOs over 60, respectively.
- $s_{\text{under 60}}$ and $s_{\text{over 60}}$ are the sample standard deviations for each age group.
- $n_{\text{under 60}}$ and $n_{\text{over 60}}$ are the sample sizes for each group.

Plugging in the values:

$$t = \frac{1930.42 - 2214.10}{\sqrt{\frac{1676.14^2}{294} + \frac{1799.35^2}{153}}} = -1.6186$$

- **Rejection Area:** For a two-sided test with a significance level of 5%, we reject the null hypothesis H_0 if the p-value is less than 0.05.
- **p-Value:** 0.1066
- **Decision:** Since the p-value of 0.1066 is greater than 0.05, we fail to reject the null hypothesis.
- **Verbal Conclusion:** Based on the results, we do not have enough statistical evidence to conclude that there is a significant difference in the average salaries of CEOs under 60 compared to those over 60. Therefore, the data suggests that the average salaries for both age groups are not significantly different.

Part C

We suspect that the sales are not related to the profits. Check this using a test for the Pearson correlation coefficient.

- **Hypotheses:**

- H_0 : There is no correlation between sales and profits.
- H_1 : There is a correlation between sales and profits.

- **Test:** Pearson correlation test to assess the linear relationship between sales and profits.

```
In [85]: # Compute the Pearson correlation coefficient and p-value
corr_coefficient, p_value = stats.pearsonr(df['sales'], df['profits'])

# Printing the results
print(f"Pearson Correlation Coefficient between Sales and Profits: {corr_coef
print(f"p-Value: {p_value:.4f}")

# Interpretation based on significance level
if p_value < 0.05:
    print("Conclusion: We reject the null hypothesis and conclude that there
else:
    print("Conclusion: We fail to reject the null hypothesis, suggesting tha
```

Pearson Correlation Coefficient between Sales and Profits: 0.7260

p-Value: 0.0000

Conclusion: We reject the null hypothesis and conclude that there is a significant correlation between sales and profits.

- **Formula for Test Statistic:** The Pearson correlation coefficient, r , is calculated as:

$$r = \frac{\sum (X - \bar{X})(Y - \bar{Y})}{\sqrt{\sum (X - \bar{X})^2} \sqrt{\sum (Y - \bar{Y})^2}}$$

where:

- X and Y are the sample values for sales and profits, respectively.
- $\bar{X}$ and $\bar{Y}$ are the means of sales and profits.
- **Rejection Area:** For a significance level of 5%, we reject H_0 if the p-value is less than 0.05.
- **Results:**
 - Pearson Correlation Coefficient between Sales and Profits: **0.7260**
 - p-Value: **0.0000**
- **Decision:** Based on the p-value (0.0), we reject the null hypothesis.

- **Verbal Conclusion:** It can be concluded that there is a statistically significant positive correlation between sales and profits. This suggests a strong linear relationship between these two variables, indicating that as sales increase, profits tend to increase as well.

Part D

Test if the fraction of CEOs older than 60 is significantly higher than 50%.

- **Hypotheses:**
 - H_0 : The proportion of CEOs over 60 is ≤ 0.5 .
 - H_1 : The proportion of CEOs over 60 is > 0.5 .
- **Test:** One-sample proportion test to see if the fraction of CEOs older than 60 is significantly higher than 50%.

```
In [86]: # Threshold proportion and count of CEOs over 60
p_0 = 0.5 # hypothesized proportion
n = len(df) # total number of CEOs
over_60_count = np.sum(df['age'] > 60) # count of CEOs older than 60
p_hat = over_60_count / n # sample proportion of CEOs over 60

# Calculate the z-statistic for the one-sample proportion test
z_statistic = (p_hat - p_0) / np.sqrt((p_0 * (1 - p_0)) / n)

# Calculate the p-value for a one-sided test
p_value = 1 - norm.cdf(z_statistic)

# Printing the results
print(f"Sample Proportion (CEOs over 60): {p_hat:.4f}")
print(f"Z-Statistic: {z_statistic:.4f}")
print(f"P-Value: {p_value:.4f}")

# Interpretation based on significance level
if p_value < 0.05:
    print("Conclusion: We reject the null hypothesis and conclude that the p
else:
    print("Conclusion: We fail to reject the null hypothesis, suggesting no
```

Sample Proportion (CEOs over 60): 0.2685

Z-Statistic: -9.7908

P-Value: 1.0000

Conclusion: We fail to reject the null hypothesis, suggesting no significant evidence that the proportion of CEOs over 60 is greater than 50%.

- **Formula for Test Statistic:**

$$z = \frac{\hat{p} - p_0}{\sqrt{\frac{p_0(1-p_0)}{n}}}$$

where:

- $\hat{p} = 0.2685$: Sample proportion of CEOs over 60.
- $p_0 = 0.5$: Hypothesized proportion.
- $n = 447$: Sample size.

Plugging in the values:

$$z = \frac{0.2685 - 0.5}{\sqrt{\frac{0.5 \times (1-0.5)}{447}}} = -9.7908$$

- **Rejection Area:** For a one-sided test at a significance level of 5%, we reject H_0 if the p-value is less than 0.05.
- **p-Value:** 1.0000, which is greater than the significance level of 0.05.
- **Decision:** Based on the p-value (1.0), we fail to reject the null hypothesis..
- **Verbal Conclusion:** This suggests that there is no significant evidence that the proportion of CEOs over 60 is greater than 50%. In fact, the sample proportion of 0.2685 (or 26.85%) indicates that CEOs over 60 make up less than half of the sample.

Part E

All the above tests rely on the normality assumption of *salary*. Check this assumption using the Kolmogorov-Smirnov test. Use only built-it functions, no manual calculations here.

- **Hypotheses:**

- H_0 : Salary is normally distributed.
- H_1 : Salary is not normally distributed.

- **Test:** Kolmogorov-Smirnov (K-S) test for normality.

```
In [87]: # Extract salary data
salary_data = df['salary']
```



```

# Perform the Kolmogorov-Smirnov test for normality
ks_statistic, p_value = kstest(salary_data, 'norm', args=(salary_data.mean(),
salary_data.std()))

# Printing the results
print(f"Kolmogorov-Smirnov Statistic: {ks_statistic:.4f}")
print(f"P-Value: {p_value:.4f}")

# Interpretation based on significance level
if p_value < 0.05:
    print("Conclusion: We reject the null hypothesis, suggesting that the salary distribution significantly deviates from normality.")
else:
    print("Conclusion: We fail to reject the null hypothesis, suggesting no significant deviation from normality.")

```

Kolmogorov-Smirnov Statistic: 0.1862

P-Value: 0.0000

Conclusion: We reject the null hypothesis, suggesting that the salary distribution significantly deviates from normality.

- **Formula for Test Statistic:** The Kolmogorov-Smirnov test statistic for normality is calculated as:

$$D = \sup_x |F_n(x) - F(x)|$$

where:

- $F_n(x)$: Empirical distribution function of the sample.
- $F(x)$: Cumulative distribution function of the normal distribution with sample mean and standard deviation.
- **Results:**
 - K-S Statistic: 0.1862
 - p-Value: 4.94×10^{-14}
- **Rejection Area:** For a significance level of 5%, we reject H_0 if the p-value is less than 0.05.
- **Decision:** Since the p-value is extremely small (less than 0.05), we reject the null hypothesis.
- **Verbal Conclusion:** This result suggests that the distribution of salaries significantly deviates from normality, implying that the salary data is not normally distributed. Additionally, based on the plot above, it is visually apparent that the salary distribution is skewed and not symmetric, further supporting the conclusion that it does not follow a normal distribution.

Task 4

The next objective is to check if the probability of a type 1 error (size of a test) is correctly attained by a simple two-sided test for the mean.

Part A

- Simulate a sample of length $n = 100$ from a normal distribution with mean $\mu_0 = 500$ and variance $\sigma^2 = 50$. (Note: you may use the transformation $X = \mu + \sigma Z$, where $Z \sim N(0, 1)$.)
- The objective is to test the null hypothesis $H_0: \mu = 500$.
- Assume that σ^2 has to be estimated.
- Calculate the test statistic using the formulas in the lecture; determine the rejection area for $\alpha = 0.04$ and decide if H_0 can be rejected.

We want to test whether the sample mean differs significantly from the hypothesized mean $\mu_0 = 500$. This is a two-sided test where the null hypothesis H_0 states that the population mean is equal to 500.

- **Hypotheses:**

- Null Hypothesis (H_0): $\mu = \mu_0 = 500$
- Alternative Hypothesis (H_1): $\mu \neq 500$

- **Significance Level:**

- The significance level is given as $\alpha = 0.04$.
- For a two-sided test, this implies that we split the significance level into two tails, with each tail covering an area of $\alpha/2 = 0.02$.

- **Test Statistic:**

- Since the variance σ^2 is not known, we estimate it using the sample standard deviation s . The test statistic V is calculated as:

$$V = \sqrt{n} \frac{|\bar{x} - \mu_0|}{s}$$

where:

- $\bar{x}$ is the sample mean,
- μ_0 is the hypothesized mean,
- s is the sample standard deviation,
- n is the sample size.

- **Rejection Area:**

- The rejection area is determined by the critical values for the two-sided test. We will reject H_0 if the absolute value of V falls outside the range of

the critical values.

- For a two-sided test with $\alpha = 0.04$, the critical values c_u and c_o are based on the t-distribution with $n - 1$ degrees of freedom (since the population standard deviation is unknown).
- We reject H_0 if:

$$V < -t_{0.02, n-1} \quad \text{or} \quad V > t_{0.02, n-1}$$

where $t_{0.02, n-1}$ is the critical value of the t-distribution at $\alpha/2 = 0.02$ with $n - 1$ degrees of freedom.

```
In [88]: # Parameters for the simulation
n = 100 # Sample size
mu_0 = 500 # Hypothesized mean
sigma_squared = 50 # Variance of the distribution
alpha = 0.04 # Significance level

# Simulate a sample
np.random.seed(0) # For reproducibility
sample = mu_0 + np.sqrt(sigma_squared) * np.random.randn(n)

# Calculate sample mean and sample standard deviation
sample_mean = np.mean(sample)
sample_std = np.std(sample, ddof=1) # Using ddof=1 for unbiased estimate

# Calculate the test statistic
t_statistic = (sample_mean - mu_0) / (sample_std / np.sqrt(n))

# Determine the rejection region for a two-sided test
critical_value = t.ppf(1 - alpha / 2, df=n-1) # t critical value for two-si
rejection_region = (-critical_value, critical_value)

# Printing the results
print(f"Sample Mean: {sample_mean:.4f}")
print(f"Sample Standard Deviation: {sample_std:.4f}")
print(f"T-Statistic: {t_statistic:.4f}")
print(f"Rejection Region: (-{critical_value:.4f}, {critical_value:.4f})")

# Interpretation based on significance level
if t_statistic < -critical_value or t_statistic > critical_value:
    print("Conclusion: We reject the null hypothesis.")
else:
    print("Conclusion: We fail to reject the null hypothesis.")
```

Sample Mean: 500.4229

Sample Standard Deviation: 7.1627

T-Statistic: 0.5904

Rejection Region: (-2.0812, 2.0812)

Conclusion: We fail to reject the null hypothesis.

We fail to reject H_0 , so it can be concluded that there is no significant evidence to suggest the sample mean is different from 500. This result aligns with our

expectations, as the data was generated from a distribution with mean $\mu_0 = 500$.

Part B

- Simulate $M = 1000$ samples of size $n = 100$ with $\mu_0 = 500$ and variance $\sigma^2 = 50$. For each sample i , run the test (using a standard function) and set $p_i = 0$ if H_0 is not rejected and $p_i = 1$ if rejected.
- Compute $\hat{\alpha} = \frac{1}{M} \sum_{i=1}^M p_i$. $\hat{\alpha}$ is the empirical confidence level (empirical size) of the test. Compare $\hat{\alpha}$ with α .
- **Question:** Do you expect the difference to be large or small and why? Relate it to the assumptions of the test.

We can expect the difference between the empirical size $\hat{\alpha}$ and the nominal size α to be small. This expectation is based on the assumptions of the test, which include:

1. **Normality of the Distribution:** Since the data is generated from a normal distribution, this aligns with the normality assumption of the t-test, making it likely that the test will yield accurate results for the type I error rate.
2. **Large Sample Size:** With a sample size of $n = 100$ per test, the estimate of the sample mean should be close to the population mean due to the law of large numbers, leading to stable and reliable t-test results.
3. **Random Sampling Variability:** Although we are using simulated samples, random variability can introduce slight fluctuations in the observed type I error rate. However, these fluctuations are expected to be minimal due to the large sample size and normal distribution.

Therefore, under these conditions, we would expect the empirical size $\hat{\alpha}$ to closely match the nominal α , with only minor differences due to sampling variability.

```
In [89]: # Parameters for the simulation
M = 1000 # Number of samples
n = 100 # Sample size
mu_0 = 500 # True mean
sigma = np.sqrt(50) # Standard deviation from variance
alpha = 0.04 # Significance level

# Store results of hypothesis tests
p_values = []

for _ in range(M):
    # Generate sample
    sample = mu_0 + sigma * np.random.randn(n)
```

```

sample_mean = np.mean(sample)
sample_std = np.std(sample, ddof=1)

# Calculate the t-statistic for the sample
t_statistic = (sample_mean - mu_0) / (sample_std / np.sqrt(n))

# Calculate the critical value for two-sided test
critical_value = t.ppf(1 - alpha / 2, df=n - 1)

# Check if the t-statistic falls outside the non-rejection region
if abs(t_statistic) > critical_value:
    p_values.append(1) # Reject H0
else:
    p_values.append(0) # Do not reject H0

# Calculate empirical size of the test (observed alpha)
empirical_alpha = np.mean(p_values)

# Printing the results
print(f"Empirical Alpha (Observed Type I Error Rate): {empirical_alpha:.4f}")
print(f"Nominal Alpha (Significance Level): {alpha:.2f}")

# Interpretation based on significance level
if empirical_alpha > alpha:
    print("The empirical size is slightly higher than the nominal size, possibly due to sample variability.")
else:
    print("The empirical size is close to or slightly lower than the nominal size.")

```

Empirical Alpha (Observed Type I Error Rate): 0.0510

Nominal Alpha (Significance Level): 0.04

The empirical size is slightly higher than the nominal size, possibly due to sample variability.

Conclusion

The empirical size of the test, $\hat{\alpha} = 0.0510$, is slightly higher than the nominal size $\alpha = 0.04$. This minor discrepancy can be attributed to random variability in the samples. However, the difference is small, which aligns with the expectations under the assumptions of the test. Given the large sample size and normal distribution assumption, we would expect $\hat{\alpha}$ to be close to the nominal α , with minor deviations due to the inherent randomness in sampling. This result suggests that the test performs as expected, and the observed type of error rate is reasonably close to the intended significance level.

Task 5

The next objective is to assess the probability of type II error (power of a test) of a goodness-of-fit test. Goodness-of-fit tests for the normal distribution are of key importance in statistics, as they allow verification of the distributional assumptions required in many models. Here, we will check the power of the

Kolmogorov-Smirnov test, i.e., whether the test can detect deviations from normality effectively.

- Simulate $M = 1000$ samples of size 100 from a t -distribution with $df = 2, \dots, 50$ degrees of freedom.
- For each sample, run the Kolmogorov-Smirnov test and count the cases where H_0 of normality is correctly rejected (for each df).
- Use this quantity to estimate the power of the test.
- Create a plot with df on the x-axis and the power of the test on the y-axis.

Note: The t -distribution converges to the normal distribution as df tends to infinity. For $df > 50$, the t -distributions are almost identical to the normal distribution.

- Analyze the plot to draw conclusions about the reliability of the Kolmogorov-Smirnov test in detecting deviations from normality across different degrees of freedom.

```
In [93]: # Parameters for the simulation
M = 1000 # Number of samples
n = 100  # Sample size for each test
degrees_of_freedom = range(2, 51) # Degrees of freedom from 2 to 50
alpha = 0.05 # Significance level for the Kolmogorov-Smirnov test

# Store power and confidence intervals for each degree of freedom
power_values = []
lower_bounds = []
upper_bounds = []

# Loop over each degree of freedom
for df in degrees_of_freedom:
    rejection_count = 0

    # Generate M samples and perform the K-S test on each
    for _ in range(M):
        # Generate a sample from the t-distribution with given df
        sample = t.rvs(df, size=n)

        # Perform the Kolmogorov-Smirnov test for normality
        ks_statistic, p_value = kstest(sample, 'norm', args=(sample.mean(),

        # Check if we reject the null hypothesis
        if p_value < alpha:
            rejection_count += 1

    # Calculate power as the proportion of rejections
    power = rejection_count / M
    power_values.append(power)

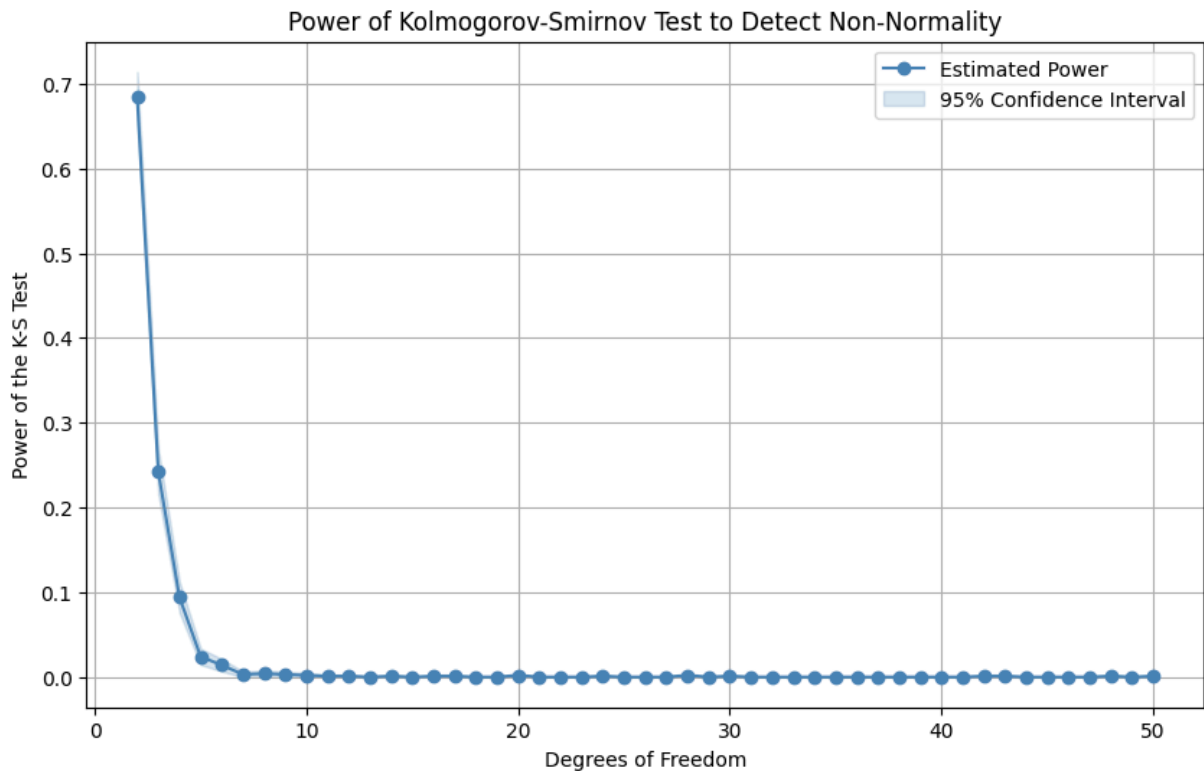
    # Calculate 95% confidence interval for the power estimate
    ci_low, ci_up = proportion_confint(count=rejection_count, nobs=M, alpha=
    lower_bounds.append(ci_low)
```

```

upper_bounds.append(ci_up)

# Plot the power of the test as a function of degrees of freedom with confidence interval
plt.figure(figsize=(10, 6))
plt.plot(degrees_of_freedom, power_values, marker='o', linestyle='--', color='blue')
plt.fill_between(degrees_of_freedom, lower_bounds, upper_bounds, color='steelblue', alpha=0.3)
plt.xlabel('Degrees of Freedom')
plt.ylabel('Power of the K-S Test')
plt.title('Power of Kolmogorov-Smirnov Test to Detect Non-Normality')
plt.legend()
plt.grid()
plt.show()

```



Conclusion

- The K-S test exhibits high power (close to 0.7) when the degrees of freedom are low, such as around $df = 2$. This suggests that the K-S test is effective at detecting deviations from normality when the sample distribution is heavily tailed, as is the case with low degrees of freedom in the t -distribution.
- As the degrees of freedom increase, the power of the test rapidly decreases and approaches zero. This is expected because the t -distribution becomes closer to the normal distribution as the degrees of freedom increase. By around $df = 10$, the power is very low, indicating that the K-S test is unlikely to detect non-normality in distributions that are nearly normal.
- For $df > 20$, the power remains close to zero, suggesting that the K-S test has little to no ability to detect deviations from normality in distributions that are

almost indistinguishable from the normal distribution. This aligns with the theoretical convergence of the t -distribution to the normal distribution as df approaches infinity.

The K-S test is reliable for detecting strong deviations from normality (such as those in distributions with heavy tails) but becomes less effective as the distribution approaches normality.

We can also extend the objective of this analysis and assess the power of different goodness-of-fit tests for detecting deviations from normality, specifically using the Kolmogorov-Smirnov (K-S) test, Shapiro-Wilk test, and Anderson-Darling test. Each test has unique properties and assumptions that make it more or less suitable depending on the degree of deviation from normality.

- **Kolmogorov-Smirnov Test:** A non-parametric test often used for normality testing. While it's versatile, it may be less powerful compared to other tests when sample sizes are small or the deviations from normality are subtle.
- **Shapiro-Wilk Test:** Known for being sensitive to deviations from normality, especially for smaller sample sizes, this test is often used for detecting departures from a normal distribution in samples of limited size.
- **Anderson-Darling Test:** An extension of the K-S test, the Anderson-Darling test gives more weight to the tails of the distribution, making it effective for detecting deviations in both the center and tails.

```
In [97]: # Parameters for the simulation
M = 1000 # Number of samples
n = 100  # Sample size for each test
degrees_of_freedom = range(2, 51) # Degrees of freedom from 2 to 50
alpha = 0.05 # Significance level for the tests

# Arrays for storing power values and confidence intervals
ks_power_values, shapiro_power_values, anderson_power_values = [], [], []
ks_lower_bounds, ks_upper_bounds = [], []
shapiro_lower_bounds, shapiro_upper_bounds = [], []
anderson_lower_bounds, anderson_upper_bounds = [], []

# Loop over each degree of freedom
for df in degrees_of_freedom:
    ks_rejections, shapiro_rejections, anderson_rejections = 0, 0, 0

    for _ in range(M):
        sample = t.rvs(df, size=n) # Generate a sample from the t-distribution

        # Kolmogorov-Smirnov test for normality
        ks_statistic, p_ks = kstest(sample, 'norm', args=(sample.mean(), sample.std()))
        if p_ks < alpha:
            ks_rejections += 1
```



```

# Shapiro-Wilk test
_, p_shapiro = shapiro(sample)
if p_shapiro < alpha:
    shapiro_rejections += 1

# Anderson-Darling test
result = anderson(sample, dist='norm')
if result.statistic > result.critical_values[2]: # Compare with 5%
    anderson_rejections += 1

# Calculate power and confidence intervals for each test
# Kolmogorov-Smirnov Test
ks_power = ks_rejections / M
ks_power_values.append(ks_power)
ks_ci_low, ks_ci_up = proportion_confint(ks_rejections, M, alpha=0.05, m
ks_lower_bounds.append(ks_ci_low)
ks_upper_bounds.append(ks_ci_up)

# Shapiro-Wilk Test
shapiro_power = shapiro_rejections / M
shapiro_power_values.append(shapiro_power)
shapiro_ci_low, shapiro_ci_up = proportion_confint(shapiro_rejections, M
shapiro_lower_bounds.append(shapiro_ci_low)
shapiro_upper_bounds.append(shapiro_ci_up)

# Anderson-Darling Test
anderson_power = anderson_rejections / M
anderson_power_values.append(anderson_power)
anderson_ci_low, anderson_ci_up = proportion_confint(anderson_rejections
anderson_lower_bounds.append(anderson_ci_low)
anderson_upper_bounds.append(anderson_ci_up)

# Plot the power of the tests as a function of degrees of freedom with confi
plt.figure(figsize=(12, 8))

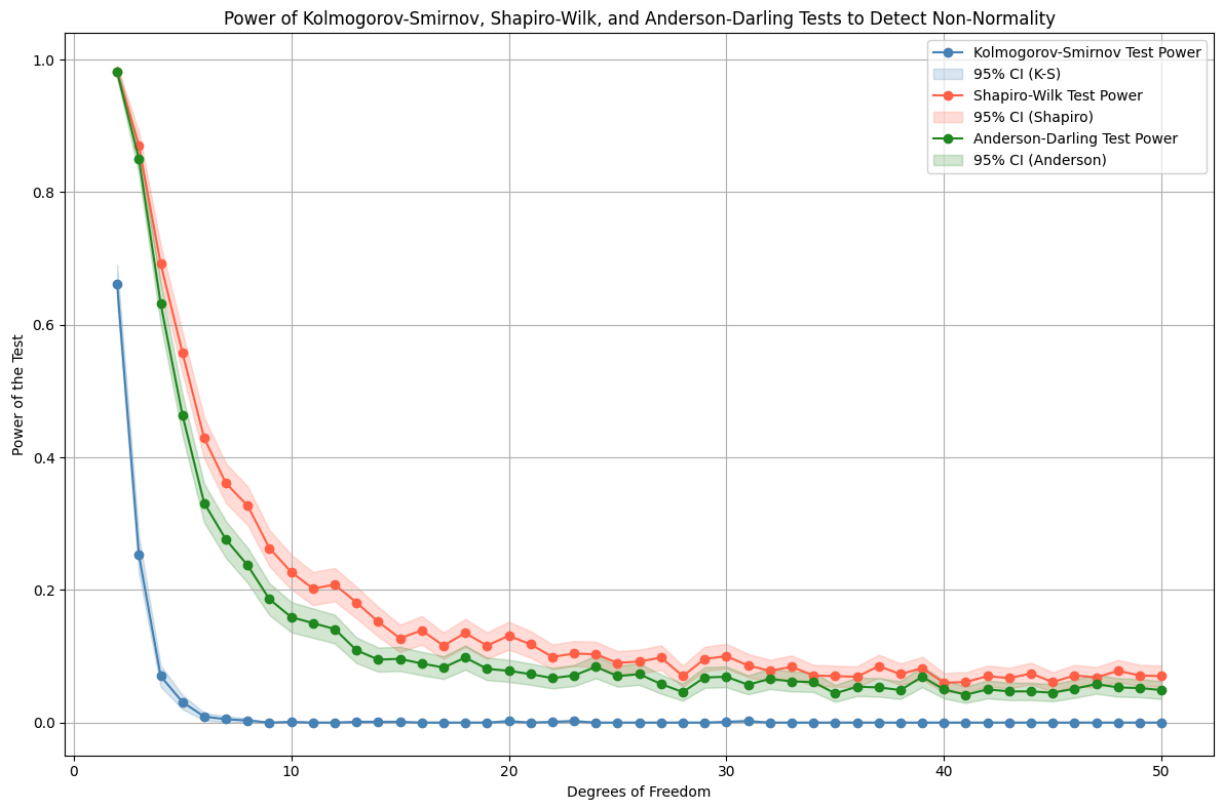
# Plot for Kolmogorov-Smirnov Test
plt.plot(degrees_of_freedom, ks_power_values, marker='o', linestyle='-', col
plt.fill_between(degrees_of_freedom, ks_lower_bounds, ks_upper_bounds, color

# Plot for Shapiro-Wilk Test
plt.plot(degrees_of_freedom, shapiro_power_values, marker='o', linestyle='-'
plt.fill_between(degrees_of_freedom, shapiro_lower_bounds, shapiro_upper_bou

# Plot for Anderson-Darling Test
plt.plot(degrees_of_freedom, anderson_power_values, marker='o', linestyle='-'
plt.fill_between(degrees_of_freedom, anderson_lower_bounds, anderson_upper_b

plt.xlabel('Degrees of Freedom')
plt.ylabel('Power of the Test')
plt.title('Power of Kolmogorov-Smirnov, Shapiro-Wilk, and Anderson-Darling T
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()

```



Conclusion

- **Shapiro-Wilk and Anderson-Darling Tests:** Both tests demonstrate high power for low df values (close to 2), indicating strong sensitivity to non-normality. However, as df increases, their power declines, approaching zero beyond $df \approx 20$. They outperform the K-S test across most df values, suggesting they are more effective for detecting subtle deviations from normality.
- **Kolmogorov-Smirnov Test:** The K-S test has significantly lower power across all df values compared to the other two tests, particularly for low df values where heavy-tailed distributions deviate most from normality. Its power quickly approaches zero, highlighting its limited effectiveness in detecting non-normality compared to the Shapiro-Wilk and Anderson-Darling tests.

The Shapiro-Wilk and Anderson-Darling tests are more reliable choices for detecting non-normality in distributions with varying degrees of freedom. The K-S test, with consistently lower power, may miss deviations that the other tests can identify.